Les DTOs (Data Transfer Object)

Table des matières

1. Introduction 2
2. Mise en place 2

1. Introduction

Lorsque notre API est en **lecture seule**, sécuriser ce qui est affiché ou non en utilisant les **groupes** est suffisant. Mais lorsque l'on ouvre la possibilité à **l'écriture**, nous avons la possibilité de mettre en place des vérifications **beaucoup plus performantes**.

Prenons un exemple : Dans notre projet Autopote, je veux donner aux utilisateurs connectés la possibilité de créer des nouvelles catégories de pièces. Les utilisateurs devront donc envoyer, en POST, quelque chose comme ceci dans le corps de la requête :

{ name : « CategorieCustom », description : « Lorem Ipsum », slug : « categorie-test » }

Mais comment je peux faire pour vérifier que les données envoyées sont bien correctes ? Qu'en est-il si l'entité cible contient des données sensibles que je ne veux pas laisser à n'importe qui ? Par exemple un utilisateur qui s'attribuerait des rôles en créant son compte.

C'est à cette problématique que répondent le DTOs, qui vont faire la partie du boulot que les Groupes de serialisation ne font pas :

- Les groupes de serialisation permettent de vérifier ce qui **sort** ou se **passe** dans l'API
- Les DTOs permettent de vérifier ce qui **entre**

2. Mise en place

Pour une fois, rien à installer, on va simplement aller créer le fichier suivant :

/src/DTO/CreateCategoryDTO.php :

```php
<?php

namespace App\DTO;

use Symfony\Component\Validator\Constraints as Assert;

class CreateCategoryDTO
{
    #[Assert\NotBlank(message: 'Le nom est obligatoire.')]
    #[Assert\Length(
        min: 2,
        max: 100,
        minMessage: 'Le nom doit faire au moins {{ limit }} caractères.',
        maxMessage: 'Le nom ne peut pas dépasser {{ limit }} caractères.'
    )]
    public string $name;

    #[Assert\Length(
        max: 500,
        maxMessage: 'La description ne peut pas dépasser {{ limit }} caractères.'
    )]
    public ?string $description = null;
}
```

Pourquoi on n'a que le **name** et la **description** ? Justement pour ne pas autoriser l'utilisateur à envoyer des données non voulues : L'id sera généré automatiquement et le slug aussi, donc on ne laisse même pas passer ces champs en entrée puisqu'on en n'a pas besoin ☺

Le DTO agit comme une passoire **AVANT MÊME** que l'API se mette à bosser, aucun risque d'avoir des ID forcés par les utilisateurs en base.

Deuxième et dernier fichier à créer : un **controleur**

Effectivement, avec ce DTO on va court-circuiter un peu le comportement prévu de API Platform, on va donc avoir besoin de s'approprier la route /api/categories en POST pour y préciser les choses au lieu de laisser API Platform gérer le comportement par défaut.

/src/Controller/CategorieController.php :

```php
<?php
// src/Controller/CategoryController.php

namespace App\Controller;

use App\DTO\CreateCategoryDTO;
use App\Entity\Category;
use Doctrine\ORM\EntityManagerInterface;
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\JsonResponse;
use Symfony\Component\HttpKernel\Attribute\MapRequestPayload;
use Symfony\Component\Routing\Attribute\Route;
use Symfony\Component\Security\Http\Attribute\IsGranted;
use Symfony\Component\String\Slugger\SluggerInterface;

class CategoryController extends AbstractController
{
    #[Route('/api/categories', methods: ['POST'])]
    #[IsGranted('ROLE_ADMIN')]
    public function create(
        #[MapRequestPayload] CreateCategoryDTO $dto,
        EntityManagerInterface $em,
        SluggerInterface $slugger
    ): JsonResponse {

        // À ce stade, $dto est déjà validé.
        // Sinon, on reçoit une 422 en réponse HTTP

        $category = new Category();
        $category->setName($dto->name);
        $category->setDescription($dto->description);

        // On génère le slug ici puisqu'on a interdit à l'utilisateur de l'envoyer
        $slug = strtolower($slugger->slug($dto->name));
        $category->setSlug($slug);

        // Et on persiste la catégorie
        $em->persist($category);
        $em->flush();

        // Puis on renvoie l'objet créé en lui passant le groupe de lecture : il saura quelles propriétés donner !
        return $this->json($category, 201, [], ['groups' => ['category:read']]);
    }
}
```

Grâce à **#[MapRequestPayload]**, le corps de la requête envoyé sur cette route est obligé de passer par le **DTO**, et c'est à ce moment là qu'il passe ou non la validation.

Côté Angular, quand on envoie un **body** mal formé sur la route, on reçoit ceci :

```
// POST /api/categories — body: { "name": "" }

// Réponse 422 :

{

  "status": 422,

  "violations": [

    {

      "propertyPath": "name",

      "message": "Le nom est obligatoire."

    }

  ]

}
```

Mais si tout va bien, on reçoit ceci :

```
// Réponse 201 :

{

  "id": 12,

  "name": "Freinage",

  "description": "Plaquettes, disques...",

  "slug": "freinage",

  "parts": []

}
```

EXERCICE

1. Créer le fichier DTO pour les marques

2. Créez le controlleur pour intercepter les requêtes en POST sur la route /api/brands

3. Créez le fichier DTO pour les pièces : Incluez tout sauf l'id, y compris la propriété $brand et la propriété $category. Attention, on ne va pas passer la catégorie entière côté Angular ! On va devoir passer l'id de la marque et de la catégorie seulement : le DTO valide l'id, le contrôlleur utilise les repositories respectif pour trouver l'objet complet et construire la relation.

4. Créez le controlleur pour intercépter les requêtes POST sur la route /api/parts

5. Côté Angular, authentifiez-vous et créez le formulaire permettant de créer des nouvelles pièces ☺